

Laboratorio 2

Reconocimiento de Entidades Nombradas con Orientación a Objetos

Objetivo

Modelar y resolver un problema real utilizando **programación orientada a objetos en Scala**, aplicando los siguientes conceptos:

- Modelado de dominios mediante **clases y jerarquías de tipos**
 - Uso de **herencia** para representar relaciones conceptuales entre entidades
 - Uso de **traits** para compartir comportamiento entre distintos tipos
 - Aplicación de **polimorfismo** para operar de forma uniforme sobre distintos objetos
 - Encapsulación de comportamiento en métodos asociados a los objetos
 - Separación clara entre **modelo de dominio**, **carga de datos** y **lógica de análisis**
-

Contexto del proyecto

En el laboratorio anterior desarrollaron un sistema que:

- Descargaba posts desde distintos subreddits de Reddit
- Procesaba el texto de cada post
- Realizaba análisis simples de palabras

En este laboratorio vamos a **reutilizar ese texto** para construir un sistema de **reconocimiento de entidades nombradas** (Named Entity Recognition, NER).

El objetivo es **identificar expresiones en el texto que correspondan a entidades conocidas y clasificarlas según su tipo**.

Ejemplo

Dado el texto:

None

"Scala fue creado en EPFL por Martin Odersky."

El sistema debe reconocer:

Texto detectado	Tipo de entidad
Scala	ProgrammingLanguage
EPFL	University
Martin Odersky	Person

Esqueleto de código provisto

Deberán descargar el esqueleto de código del siguiente link:

<https://drive.google.com/file/d/1zrprpPURgQcqZy2rR9Sy6JnPx2KTS7U9/view?usp=sharing>

El proyecto ya cuenta con la siguiente estructura:

None

lab2/

├─ build.sbt

├─ data/

| ├─ people.txt # nombres de personas conocidas

| ├─ universities.txt # nombres de universidades

| ├─ languages.txt # lenguajes de programación

| ├─ organizations.txt # empresas y organizaciones

| └─ places.txt # lugares relevantes

└─ src/main/scala/

— NamedEntity.scala	# jerarquía de entidades (completar)
— Dictionary.scala (implementar)	# carga de diccionarios
— Analyzer.scala	# detección y conteo (implementar)
— Formatters.scala (implementar)	# formateo de resultados
— FileIO.scala	# E/S de archivos y feeds (provisto)
└ Main.scala	# punto de entrada (integrar)

Qué está provisto

Archivo	Estado	Descripción
FileIO.scala	Completo	Descarga feeds, extrae títulos, lee archivos
NamedEntity.scala	Esqueleto	Clase base abstracta (completar jerarquía)
Dictionary.scala	Esqueleto	Estructura con ??? para que completen
Analyzer.scala	Esqueleto	Estructura con ??? para que completen
Formatters.scala	Esqueleto	Estructura con ??? para que completen
Main.scala	Esqueleto	Flujo integrador con ??? para que completen
data/*.txt	Completo	Diccionarios de entidades con entradas de ejemplo

Descripción del sistema de diccionarios

Un **diccionario** (también llamado *gazetteer*) es un archivo de texto plano donde cada línea contiene el nombre de una entidad conocida de ese tipo.

data/people.txt

```
None
Martin Odersky
Alan Turing
Ada Lovelace
```

data/universities.txt

```
None
EPFL
MIT
Stanford
```

data/languages.txt

```
None
Scala
Python
Haskell
```

El sistema cargará estos archivos al iniciar y los usará para identificar entidades en el texto de los posts.

Ejercicio 1 — Modelar la jerarquía de entidades

Completar el archivo `NamedEntity.scala` implementando las clases de la siguiente jerarquía:

None

```
NamedEntity      ← clase base abstracta (provista)
├─ Person        ← clase concreta
├─ Organization   ← clase concreta
│   └─ University ← subclase de Organization
├─ Place         ← clase concreta
└─ Technology    ← clase concreta
    └─ ProgrammingLanguage ← subclase de Technology
```

Requisitos:

- Cada clase debe extender correctamente su clase padre.
- Cada clase concreta debe implementar el método abstracto `entityType`.
- No modificar la clase base `NamedEntity`.

Verificación: El siguiente código debe compilar y producir la salida indicada:

None

```
val entities: List[NamedEntity] = List(
    new Person("Alan Turing"),
    new University("MIT"),
    new ProgrammingLanguage("Scala"),
    new Place("San Francisco")
)

entities.foreach(e => println(e.describe))
```

Salida esperada:

None

```
[Person] Alan Turing
[University] MIT
```

[ProgrammingLanguage] Scala

[Place] San Francisco

Reflexión: ¿Por qué `describe` funciona correctamente para todos los tipos sin necesidad de redefinirlo en cada subclase?

Ejercicio 2 — Cargar diccionarios

Implementar el objeto `Dictionary` en `Dictionary.scala`.

- `loadFromFile(filePath, entityType)`

Lee un archivo de diccionario línea por línea y retorna una `List[NamedEntity]` donde cada línea se convierte en una instancia de la clase correspondiente según `entityType`.

- `loadAll()`

Carga todos los diccionarios disponibles y combina los resultados en una única lista:

Archivo	Tipo
data/people.txt	"Person"
data/universities.txt	"University"
data/languages.txt	"ProgrammingLanguage"
data/organizations.txt	"Organization"
data/places.txt	"Place"

Verificación:

None

```
val dict = Dictionary.loadAll()
```

```
println(s"Total de entidades: ${dict.size}")

dict.filter(_.entityType == "Person").foreach(p =>
println(p.describe))
```

Ejercicio 3 — Detectar entidades en el texto

Implementar `Analyzer.detectEntities(text, dictionary)` en `Analyzer.scala`.

Dado un texto libre y una lista de entidades conocidas, retornar la sublista de entidades cuyo texto aparece mencionado en el texto analizado.

Indicaciones:

- Recorrer el diccionario y conservar sólo las entidades cuyo `text` aparece en el texto del post.
- Una entidad puede aparecer más de una vez en el texto, pero debe retornarse una sola vez.

Ejemplo:

```
None
val text = "Scala fue creado en EPFL por Martin Odersky"

val dict = Dictionary.loadAll()

val found = Analyzer.detectEntities(text, dict)

found.foreach(e => println(e.describe))
```

Salida esperada:

```
None
[ProgrammingLanguage] Scala

[University] EPFL

[Person] Martin Odersky
```

Ejercicio 4 — Usar polimorfismo para el informe

Implementar `Formatters.formatNERResult(postTitle, entities)` en `Formatters.scala`.

El método debe retornar un bloque de texto con el título del post y la lista de entidades detectadas.

Ejemplo de salida:

None

Post: "Scala 3 released at EPFL by Martin Odersky"

Entidades detectadas:

[ProgrammingLanguage] Scala

[University] EPFL

[Person] Martin Odersky

Si no se detectaron entidades:

None

Post: "How do I center a div?"

(sin entidades detectadas)

Reflexión: ¿Qué ventaja tiene usar métodos de forma polimórfica respecto a escribir cada caso que distinga cada subclase?

Ejercicio 5 — Contar entidades por tipo

`Analyzer.countByType(entities)`

Dado una lista de entidades detectadas, retornar un `Map[String, Int]` que indique cuántas entidades hay de cada tipo.

Ejemplo:

None

```
val entities = List(  
    new Person("Alan Turing"),  
    new ProgrammingLanguage("Scala"),  
    new Person("Ada Lovelace"),  
    new University("MIT")  
)  
  
val counts = Analyzer.countByType(entities)  
  
// Map("Person" -> 2, "ProgrammingLanguage" -> 1, "University" ->  
1)
```

`Formatters.formatEntityStats(counts)`

Formatear el mapa de conteos ordenado de mayor a menor cantidad.

Salida esperada:

None

```
=== Estadísticas de entidades ===  
  
Person: 5  
  
ProgrammingLanguage: 3  
  
Organization: 2  
  
University: 2
```

Ejercicio 6 — Integración del sistema completo

Completar `Main.scala` para que el sistema ejecute el flujo completo:

1. Cargar el diccionario
2. Descargar los posts de Reddit
3. Extraer los títulos
4. Para cada título, detectar entidades y mostrar el resultado

5. Al final, mostrar las estadísticas globales

Ejemplo de salida completa:

None

Diccionario cargado: 54 entidades.

Descargando posts de:

<https://www.reddit.com/r/scala/.json?count=10>

Post: "Scala 3.3 LTS released"

Entidades detectadas:

[ProgrammingLanguage] Scala

Post: "Martin Odersky at EPFL talks about Scala's future"

Entidades detectadas:

[Person] Martin Odersky

[University] EPFL

[ProgrammingLanguage] Scala

...

=== Estadísticas de entidades ===

ProgrammingLanguage: 8

Person: 3

University: 2

Punto estrella — Estadísticas jerárquicas

Extender el sistema para producir estadísticas **agregadas por jerarquía**: cada tipo padre debe mostrar el total acumulado de él y sus subtipos.

Ejemplo:

```
None
Technology: 8
  ProgrammingLanguage: 8

Organization: 5
  University: 2
  (Organization directa): 3
```

Esto requiere aprovechar la jerarquía de clases: una **University** es también una **Organization**, y una **ProgrammingLanguage** es también una **Technology**.

Cómo compilar y ejecutar

```
None
# Compilar
sbt compile

# Ejecutar
sbt run

# Modo interactivo
sbt
```

```
> run
```

Criterios de evaluación

Modelado orientado a objetos

- Uso correcto de herencia para expresar relaciones entre tipos
- Uso adecuado de clases abstractas y concretas
- Aplicación de traits cuando corresponda

Polimorfismo

- Uso de métodos comunes (`entityType`, `describe`) para operar sobre una lista heterogénea de entidades
- Ausencia de `if` o `match` sobre tipos donde el polimorfismo es suficiente

Encapsulación

- El comportamiento asociado a una entidad está dentro de la clase correspondiente
- Separación clara entre carga de datos, modelo de entidades y lógica de análisis

Correctitud

- El sistema detecta correctamente las entidades presentes en los posts
- Las estadísticas reflejan los datos detectados

Objetivo pedagógico

Este laboratorio busca que comprendan cuándo y por qué modelar con objetos es más apropiado que hacerlo con funciones. En conjunto con el laboratorio anterior, deben poder comparar ambos enfoques:

Paradigma funcional

Paradigma orientado a objetos

Funciones que transforman datos	Objetos que encapsulan comportamiento
---------------------------------	---------------------------------------

Composición de funciones	Jerarquías de clases
--------------------------	----------------------

Transformaciones de colecciones	Colaboración entre objetos
---------------------------------	----------------------------

Datos separados del comportamiento	Datos y comportamiento unidos
------------------------------------	-------------------------------

La pregunta clave no es cuál paradigma es "mejor", sino cuál **se adapta mejor al problema** en cada contexto.